Experiment 7

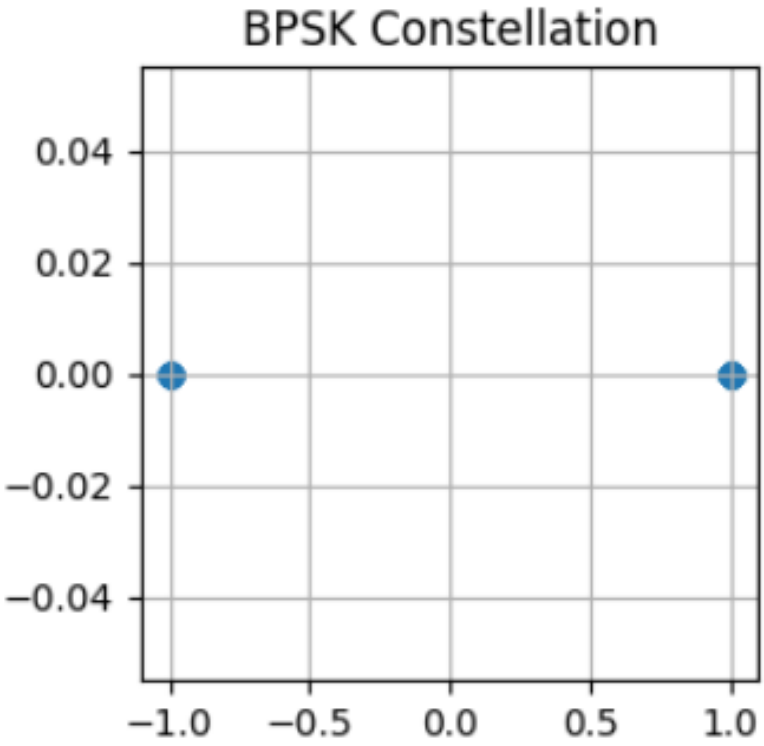
1 Python Code

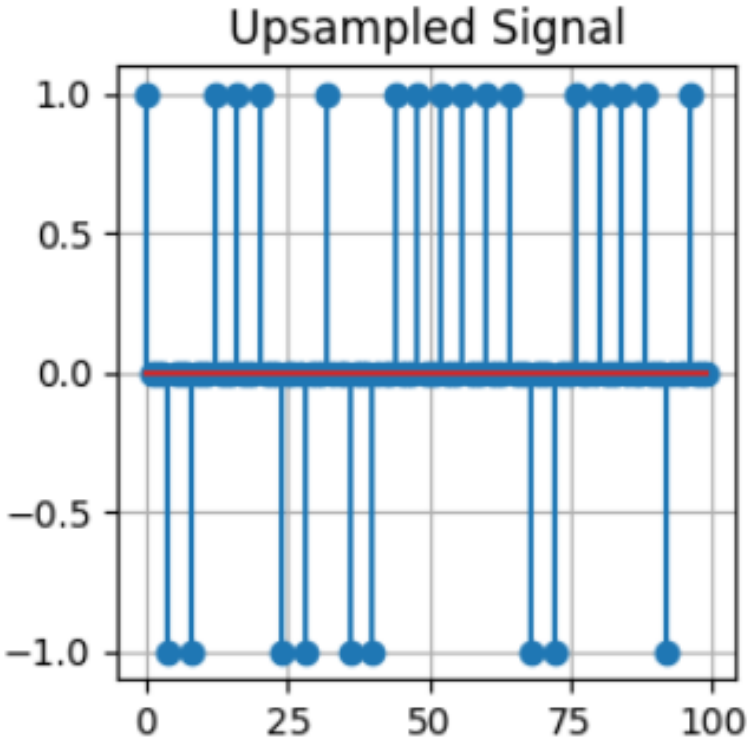
```
import numpy as np
import matplotlib.pyplot as plt
import cv2

L = 4
T_sym = 1
N_sym = 8
beta = 0.5
SNR_dB_range = np.array([0, 2, 4, 6, 8, 10, 14, 18])

img = cv2.imread('cameraman.png', cv2.IMREAD_GRAYSCALE)
img = cv2.resize(img, (256, 256))

bits = np.unpackbits(img.flatten())
a_i = 2 * bits.astype(float) - 1

plt.figure(figsize=(3, 3))
plt.scatter(a_i[:100], np.zeros(100))
plt.title("BPSK Constellation")
plt.grid(True)
plt.show()

u_n = np.zeros(len(a_i) * L)
u_n[::L] = a_i

plt.figure(figsize=(3, 3))
plt.stem(u_n[:100])
plt.title("Upsampled Signal")
plt.grid(True)
plt.show()

t = np.arange(-N_sym/2, N_sym/2 + 1/L, 1/L)

def get_srrc(t, beta, Ts):
    p = np.zeros(len(t))
```

```
    for i in range(len(t)):
        ti = t[i]
        if ti == 0.0:
            p[i] = (1/np.sqrt(Ts)) * (1 + beta*(4/np.pi - 1))
        elif np.abs(ti) == Ts/(4*beta):
            p[i] = (beta/np.sqrt(2*Ts)) * (
                (1 + 2/np.pi)*np.sin(np.pi/(4*beta)) +
                (1 - 2/np.pi)*np.cos(np.pi/(4*beta))
            )
        else:
            num = (np.sin(np.pi*ti*(1-beta)/Ts) +
                   (4*beta*ti/Ts) * np.cos(np.pi*ti*(1+beta)/Ts))
            den = (np.pi*ti/Ts) * (1 - (4*beta*ti/Ts)**2)
            p[i] = (1/np.sqrt(Ts)) * (num/den)
    return p

p_n = get_srrc(t, beta, T_sym)

plt.plot(p_n)
plt.title("SRRC Pulse")
plt.show()

s_n = np.convolve(u_n, p_n, mode='full')

plt.figure(figsize=(3, 3))
plt.stem(s_n[:100])
plt.title("Transmitted Signal")
plt.grid(True)
plt.show()

N_filter = len(p_n)
delta = (N_filter - 1) // 2
total_delay = 2 * delta

avg_pwr_s = np.mean(s_n**2)

ber_list = []
reconstructed_images = []

for snr_db in SNR_dB_range:
    snr_lin = 10**(snr_db / 10)
    p_noise = avg_pwr_s / snr_lin

    ni = np.random.normal(0,1,s_n.shape)
    nq = np.random.normal(0,1,s_n.shape)

    noise = np.sqrt(p_noise/2) * (ni + 1j*nq)
```

```
    r_n = s_n + noise

    y_n = np.convolve(r_n, p_n, mode='full')

    a_hat = y_n[total_delay : total_delay + len(a_i)*L : L]

    bits_hat = (a_hat[:len(bits)] >= 0).astype(int)

    err = np.sum(bits != bits_hat)
    ber_list.append(err / len(bits))

    rec_img = np.packbits(bits_hat).reshape(256, 256)
    reconstructed_images.append(rec_img)

    print(f"SNR: {snr_db} dB | BER: {ber_list[-1]}")

plt.figure(figsize=(16, 12))

plt.subplot(3, 4, 1)
plt.plot(t, p_n)
plt.title("SRRC Pulse Shape")
plt.grid(True)

plt.subplot(3, 4, 2)
plt.semilogy(SNR_dB_range, ber_list, 'o-')
plt.title("BER vs SNR")
plt.grid(True)

plt.subplot(3, 4, 3)
plt.imshow(img, cmap='gray')
plt.title("Original Image")

for i, snr_db in enumerate(SNR_dB_range):
    plt.subplot(3, 4, i + 4)
    plt.imshow(reconstructed_images[i], cmap='gray')
    plt.title(f"SNR = {snr_db} dB")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

2 Reconstructed Images





```
SNR:  0 dB | BER: 0.002254
SNR:  2 dB | BER: 0.000200
SNR:  4 dB | BER: 0.000002
SNR:  6 dB | BER: 0.000000
SNR:  8 dB | BER: 0.000000
SNR: 10 dB | BER: 0.000000
SNR: 14 dB | BER: 0.000000
SNR: 18 dB | BER: 0.000000
```